

Project 1 — SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint

Title: SupportDesk AI Chat And Auto Triage Any Customer Complaint

Type: Multi-agent (2 agents) · **Stack:** Python, raw OpenAI SDK (no agent framework, on purpose) · **Level:** Beginner

Tagline: A 2-agent support desk you run in your terminal — a Concierge agent that just chats, and a Triage agent that turns a free-text complaint into a clean, structured ticket.

This is the **reading guide** — everything you need to build the project yourself: the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, [README.md](#), is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/`.

Charter (what this project is)

A terminal app with two agents working together: Concierge (talks to the user) and Triage (turns a complaint into structured data). A router sends each message to the right one. No agent framework yet — just the raw OpenAI SDK. This project proves two things with real code: (1) you can call an LLM correctly, and (2) you understand what "multi-agent" really means, at its simplest. It also proves a third, quieter thing: you can lay out a Python project the way a real team would.

Jump to: [Read first](#) · [Project structure](#) · [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#) · [Checklist](#) · [Publish](#)

The Story — what you're actually building

Imagine your company gets support messages all day — some people just want to chat or ask a quick question, others are genuinely upset about something broken and need a proper ticket filed. Right now a person reads every single message and decides which is which by hand. That's slow, and it doesn't scale.

You're building a small terminal app that does both jobs for you. One part, the **Concierge**, is a normal chat agent — it holds a real conversation, remembers what was said earlier, and replies

naturally. The other part, the **Triage** agent, has a narrower, stricter job: read a messy, free-text complaint and turn it into a clean record a support system could actually use — who it's from, what kind of problem it is, how urgent it is, and a one-line summary. A small **router** sits in front of both and decides, automatically, which one should handle each message — so nobody has to say "this one's a complaint" out loud.

Each Step below builds one piece of that, in order: Step 1 proves you can talk to the model at all. Step 2 turns that into a real, remembering conversation. Step 3 adds the Triage agent and the error handling that keeps the app from crashing. Step 4 adds the router that ties Concierge and Triage together — the actual "multi-agent" part.

Before you read further — think about it yourself: if you were building this by hand, what's the very first thing you'd need to prove works, before writing any chat logic at all? And once you have two different jobs (chatting, and filing a ticket), how would your code decide, on its own, which one a given message needs?

What you're actually building, in one line: a terminal app where a router looks at each message and sends it to one of two agents — a chat agent, or a data-extraction agent.

Why this needs to exist: a real support inbox mixes casual questions with real complaints, and paying a person to sort every message by hand does not scale.

When you'd reach for this at a real job: when a product gets more than one kind of incoming message, and nobody wants to force the user to pick a category — a support inbox, a multi-purpose chatbot, any intake form.

How it works, mechanically: the router asks the model one small, focused question ("concierge or triage?") and gets back a checked answer. Then the matching agent function runs. The two agents never talk to each other — the router just picks one per message.

Why not one big prompt that does both? One prompt trying to do open-ended chat *and* strict structured output tends to do both worse — the model gets confused about which mode it's in. **Why not make the user type `/chat` or `/extract`?** That pushes the sorting work onto the user; the whole point is that the system figures it out.

Where This Fits in the 5-Project Arc

Every project in this curriculum is a real multi-agent system. What changes is *how many* agents there are, *how* they're built, and *how* they work together:

Project 1 (you are here)	→ 2 agents, Python router, raw SDK (Concierge + Triage)
Project 2	→ 2 agents, hand-built agent loop, tools

	(Worker + Verifier)
Project 3	→ 3 agents, LangGraph, RAG + human approval (Retriever + Reasoner + Approval)
Project 4	→ 4-5 agents, one Supervisor directs the rest (Supervisor + Research/Analysis/Writer/Reviewer)
Project 5	→ Project 4, made production-ready (API, database, Docker, testing)

Project 1's 2 agents are kept simple on purpose (no tools, no framework) so the only new multi-agent idea is "a router sends work to the right agent."

What to Read First (docs this project uses)

Everything this project needs was taught in Docs 01-04. Use this table when a step says "Read first", or whenever an idea feels unfamiliar:

You need to understand...	Read this	Used in
<code>venv</code> , <code>pip</code> , <code>requirements.txt</code>	Doc01 — Virtual environments	Setup
<code>.env</code> , never committing secrets	Doc01 — .env files	Setup, Step 1
<code>load_config()</code> , <code>MissingConfigError</code>	Doc01 — Errors + Doc01 Build Task	Step 1
<code>get_logger()</code> , why not <code>print()</code>	Doc01 — Logging	Step 1 onward
The client and the message list	Doc04 — The client	Step 1
Why the model forgets everything	Doc03 — No memory	Step 2
Tokens and the "4 characters" rule	Doc03 — Tokens	Step 2
The context window limit	Doc03 — Context window	Step 2
Streaming a reply	Doc04 — Streaming	Step 2
Pydantic schemas, <code>Literal</code> , <code>X \ </code> , <code>None</code>	Doc04 — Structured output	Steps 3, 4
"Valid JSON" vs. "the right shape"	Doc02 — JSON	Step 3
Which errors to retry, which not	Doc04 — Error types + Doc02 — Retrying	Step 3
The whole app, as a practice run	Doc04 Build Task	All steps
Routed vs. supervisor patterns	Doc11 — Patterns (look ahead — just the Sequential and Supervisor rows)	Step 4

`config.py`, `exceptions.py` and `logger.py` in this project are **Doc01's Build Task files**, **unchanged** — only their location and import lines change (see below). `send_message()` and `stream_message()` are **Doc04's Build Task functions**, with the small changes each step points out.

Project Structure (the professional layout)

A general AI-project template has many folders: `api/`, `rag/`, `chains/`, `notebooks/`, `docker/`, and more. A professional doesn't copy all of them — they keep **only the folders this project has a real job for**, and add the rest the day a real need appears. Below is the right-sized layout for this project, what each part is for, and what was left out on purpose.

Names: the repo folder vs. the Python package

- **Repo folder:** `SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/` — already created for you. This is the exact name you give the GitHub repo, so the folder on your machine and the repo online always match.
- **Python package:** `supportdesk` (inside `src/`). Python import names can't contain dashes or capital letters, so the code lives under a short, lowercase name: `from supportdesk.config import load_config`.
- **Files:** `snake_case.py`, named for the one job they do (`history.py`, not `utils2.py`). Test files start with `test_`, so anyone can spot them at a glance.
- **Prompt files:** named after the agent that uses them (`trriage.txt` belongs to `agents/trriage.py`).

The final layout (after Step 4)

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├─ README.md           the repo's front page (copied in at the end)
├─ LICENSE             lets others reuse your code (MIT)
├─ .gitignore          keeps .env, .venv/, __pycache__/ out of git
├─ .env.example        the setting names, no real values
├─ .env                your real key – never committed
├─ pyproject.toml      makes src/supportdesk an installable package
├─ requirements.txt    the 3 libraries, for pip install -r
├─ src/
│   └─ supportdesk/
│       ├── __init__.py
│       ├── main.py     the terminal loop – the file you run
│       ├── config.py   Config, load_config()           (Doc01)
│       ├── exceptions.py MissingConfigError       (Doc01)
│       ├── schemas.py  SupportTicket, RouteDecision
│       └─ models/
```


just claim.

- **prompts/** — the system prompts, as plain `.txt` files. **Why:** you can reword how an agent behaves without touching Python, and a reviewer can read every prompt in one folder. A prompt change shows up in git as its own clean diff.
- **services/** — the "business flow": take a message, ask the router, call the right agent. **Why:** `main.py` only reads the keyboard and prints. The flow itself lives here, so Project 5 can call the same `handle_message()` from a web API with no terminal code.
- **utils/** — small helpers that aren't about one agent: logging, history trimming, prompt loading. **Why:** shared by several files; putting them in one place stops copy-paste.
- **tests/unit/** — tests that run with no API key, no network, in under a second. **Why:** you can run them after every change, and a CI server could too (Doc12-13).
- **tests/integration/** — tests that call the real API. **Why:** kept separate because they cost money, need a key, and are slower — you run them on purpose, not by accident.
- **scripts/** — small tools you run by hand, which are not part of the app. **Why:** `triage_samples.py` prints tickets for a *person* to judge (e.g. after rewording `triage.txt`). That's a tool, not a test and not a feature.
- **data/sample/** — the sample messages, as JSON. **Why:** the same inputs every time, shared by the script and the tests, instead of messages retyped by hand in 3 places. (Doc02 taught reading JSON.)

Why each root file exists

- `README.md` — what someone sees first on GitHub: what it is, how to run it.
- `LICENSE` — without one, nobody is legally allowed to reuse your code. GitHub can add MIT for you when you create the repo.
- `.gitignore` — stops secrets (`.env`), the venv and Python's cache files from being pushed.
- `.env.example` — tells a teammate which settings to create, without any real value.
- `pyproject.toml` — the modern standard file that describes the package: its name, its libraries, and the `supportdesk` command.
- `requirements.txt` — the library list for a plain `pip install -r requirements.txt` (the Doc01 habit).

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py`. It marks the folder as a package, so `supportdesk.agents.router` is importable.
- `pip install -e .` (run once, in Setup) installs *your own* package into the venv. `-e` means "editable": Python reads your files straight from `src/`, so every edit is live — no reinstall.
- After that, every file imports the same way, from any folder: `from supportdesk.utils.logger import get_logger`. No `sys.path` tricks, no "run it from this exact folder" rules.

- Run the app with `python -m supportdesk.main` (or just `supportdesk` — the command `pyproject.toml` creates).
- Always run commands from the **repo root**. The prompt loader, the tests and the script all use paths like `data/sample/messages.json` that start there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
<code>api/</code> , <code>routes.py</code>	This is a terminal app, no web server	Project 5 (Doc12)
<code>rag/</code> , <code>vectorstore.py</code>	No documents to search	Project 3 (Doc08)
<code>chains/</code>	No LangChain yet — raw SDK on purpose	Doc05, Project 3
<code>configs/*.yaml</code>	2 settings fit in <code>.env</code> ; YAML needs a new library	When settings grow
<code>notebooks/</code>	Nothing to explore or plot	Only if you do experiments
<code>Dockerfile</code> , <code>docker-compose.yml</code>	Nothing to deploy yet	Project 5 (Doc12)
<code>Makefile</code>	3 short commands, all listed in the README	Optional, any time
<code>.github/workflows/</code>	CI is taught later	Doc12-13
<code>tests/conftest.py</code>	No pytest yet — plain test scripts, like Doc04	Doc13
<code>docs/</code>	One README is enough for this size	Bigger projects

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, <code>src/supportdesk/__init__.py</code>	—
1	<code>config.py</code> , <code>exceptions.py</code> , <code>utils/logger.py</code> , <code>models/llm.py</code> , <code>main.py</code>	—
2	<code>utils/history.py</code> , <code>utils/prompts.py</code> , <code>prompts/concierge.txt</code> , <code>tests/unit/test_history.py</code>	<code>models/llm.py</code> , <code>main.py</code>
3	<code>schemas.py</code> , <code>agents/triage.py</code> , <code>prompts/triage.txt</code> , <code>data/sample/messages.json</code> , <code>scripts/triage_samples.py</code> ,	<code>main.py</code>

Step	New files	Changed files
	tests/unit/test_schemas.py , tests/integration/test_live_agents.py	
4	agents/router.py , agents/concierge.py , prompts/router.txt , services/desk_service.py	schemas.py , main.py , messages.json , both test files

Setup (do this once, before Step 1)

Read first: [Doc01 — Virtual environments](#), [Doc01 — .env files](#).

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`) that's:

```
cd project_1_supportdesk_chat_and_triage
cd SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets – never pushed
.env
# the virtual environment – rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# created by "pip install -e ."
*.egg-info/
```

3. Create `.env` (your real key) and `.env.example` (same names, no values):

```
# .env – never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example – commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
```

```
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt` :

```
openai>=1.40
pydantic>=2.0
python-dotenv>=1.0
```

`>=` means "this version or newer" — the oldest versions this code is known to work with.

5. Create `pyproject.toml` — it tells `pip` your package's name, where its code lives, and what it needs:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "supportdesk"
version = "0.1.0"
description = "A 2-agent terminal support desk: Concierge + Triage."
requires-python = ">=3.10"
dependencies = [
    "openai>=1.40",
    "pydantic>=2.0",
    "python-dotenv>=1.0",
]

[project.scripts]
# after "pip install -e .", typing "supportdesk" runs main()
supportdesk = "supportdesk.main:main"

[tool.setuptools.packages.find]
# the package code lives under src/, not in the repo root
where = ["src"]
```

6. Create the package folder with an empty `src/supportdesk/__init__.py` , then install:

```
pip install -r requirements.txt
pip install -e .
```

Check it worked: `python -c "import supportdesk; print('ok')"` prints `ok` from any folder. (The `supportdesk` command appears now, but only works once Step 1's `main.py` exists.)

Optional — real test complaints: for realistic messy input for Step 3, you can pull a few rows from the [CFPB Consumer Complaint Database](#) — a free, public dataset of real complaints. Not required — the 3 inputs below work fine.

A Real Example (so this isn't just theory)

Scenario: the user describes a problem in plain text. Your code turns it into a `SupportTicket` :

```
customer_name: str | None
issue_category: Literal["billing", "technical", "account_access", "other"]
urgency: Literal["low", "medium", "high"]
summary: str          # one sentence describing the issue
```

Test these 3 inputs (Step 3 saves them in `data/sample/messages.json`, so you reuse the same ones every time):

1. "Hi, I'm Sarah Khan. I was charged twice for my subscription this month and I need it fixed today." → `issue_category="billing"`, `urgency="high"`, `customer_name="Sarah Khan"`.
2. "can't log into my account, tried resetting password twice, still nothing" → `issue_category="account_access"`, `customer_name=None` (no name was given — this checks your code doesn't make one up).
3. "just wondering what your refund policy is, no rush" → `issue_category` could be "other" or "billing" — a judgment call; decide, and be ready to explain. `urgency="low"`.

Why input 2 matters: a missing piece of information must stay empty (`None`) instead of the model inventing a name. This is a real problem you'll actually hit.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** most "I called an LLM" demos don't hold a real conversation, don't produce checked structured data, and don't have more than one role. This one does all three — with no framework, in a real package layout.
- **Why it matters:** it shows you understand what happens underneath the tools you'll use later.
- **When you'd build something like this at a job:** any small internal tool or first version of a product. A lot of real features are exactly this: a small router in front of two or three well-defined model calls.
- **How it's built:** you type a message → the router picks Concierge or Triage → Concierge remembers and streams, Triage extracts once. Every likely failure is handled on purpose.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The conversation gets too long for the model	Estimate tokens before each call and drop the oldest turns first (Step 2's <code>trim_history()</code>)
The structured-output call doesn't match your schema	Catch Pydantic's <code>ValidationError</code> by name and tell the user — never pass bad data on
You find out the API key is wrong mid-chat	Test the key with one 1-token call at startup (Step 1)
The router picks the wrong agent	Every decision is logged with a reason (Step 4) — read the log, then reword <code>prompts/router.txt</code>
<code>ModuleNotFoundError: No module named 'supportdesk'</code>	The venv isn't active, or you skipped <code>pip install -e .</code>

Built During These Documents

[01_python_foundations](#) → [02_apis_http_json](#) → [03_llm_fundamentals](#) → [04_openai_api](#)

Plan Before You Code

See [15_five_projects_index](#): write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks yourself, before you open your editor.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers one message, Step 2 is a chat app, Step 3 adds tickets, Step 4 is the full 2-agent desk. Each step only adds or changes a few files — nothing is thrown away. Each step says what to build, which files it adds, why they exist, and how to check it works. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command in the steps runs from the **repo root** (`SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/`) with the venv active.

Step 1 — A Script That Sends One Message and Prints the Reply

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 1 of 4

- **Where you are now:** Setup is done — the repo folder, venv, `pyproject.toml` , `.env` , and an empty `src/supportdesk/` package. Nothing runs yet.
- **What's new in this step:** Doc01's config/logging files, `models/llm.py` (the only file that talks to OpenAI), and a `main.py` that sends one message.

- **Why:** before any chat or agents, prove the foundations work together — settings load, logging works, the key is valid, one call comes back. If they don't, you find out now, alone, not mixed into Step 3's logic.
- **How:** `load_config()` → `get_logger()` → `create_client()` → `check_api_key_at_startup()` (a 1-token call) → `send_message()` → print.
- **Working after this step:** `python -m supportdesk.main` prints the model's answer to "What's 2+2?".

When you'll hit this for real: "prove the connection works" is the first step on *any* new AI project, before any feature.

Read first: [Doc01 Build Task](#) (config + logging), [Doc04 — The client and the list of messages](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do:

1. Bring in Doc01's `exceptions.py`, `config.py` and `logging_setup.py` as `src/supportdesk/exceptions.py`, `src/supportdesk/config.py` and `src/supportdesk/utils/logger.py`. Change only the import lines (`from supportdesk.exceptions import MissingConfigError`) — nothing else.
2. In `models/llm.py`, write `create_client(config)`, `check_api_key_at_startup(client)` (one 1-token call; `False` on `AuthenticationError`) and Doc04's `send_message(client, history, user_input)`.
3. In `main.py`, write `main()`: load config, get a logger, build the client, check the key, send "What's 2+2?", print the reply.
4. Run `python -m supportdesk.main`. Then put a fake key in `.env` and check you get one clear line, not a traceback.

Your files after Step 1:

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├─ (Setup's root files)
├─ src/supportdesk/
│   ├── __init__.py
│   ├── main.py          (new) sends one message, prints the reply
│   ├── config.py        (new) Config, require_env(), load_config()
│   ├── exceptions.py    (new) MissingConfigError
│   └─ models/
│       ├── __init__.py  (new)
│       └─ llm.py        (new) create_client(), check_api_key_at_startup(),
│                           send_message()
│   └─ utils/
│       ├── __init__.py  (new)
│       └─ logger.py     (new) get_logger()
```

Why these files:

- `config.py` / `exceptions.py` / `utils/logger.py` — Doc01's foundation, so settings and logging work exactly as in every other document.
- `models/llm.py` — the only file that touches the OpenAI SDK; every later file calls these functions instead.
- `main.py` — the entry point; for now it just proves the pieces connect.

Step 2 — A Real Terminal Chat With Memory and Live Streaming

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 2 of 4

- **Where you are now:** after Step 1 the app answers **one** fixed message and exits. It has settings, logging, a key check and `send_message()`.
- **What's new in this step:** a chat loop, a growing `history` list (the memory), `stream_message()`, a system prompt in `prompts/concierge.txt`, and `utils/history.py` to trim long chats. Plus the first unit test.
- **What stays the same:** `config.py`, `exceptions.py`, `utils/logger.py`, `create_client()`, `check_api_key_at_startup()`, `send_message()`.
- **Why:** the model remembers nothing between calls (Doc03), so the app must keep and resend the conversation itself. Streaming makes replies appear as they're written. Trimming stops a long chat from going over the context limit.
- **How:** each turn: read input → `trim_history()` → `stream_message()`, which appends both the user turn and the reply to `history`.
- **Working after this step:** a real terminal chat that remembers what you said earlier.

When you'll hit this for real: every chat feature you'll ever build starts with exactly this — a growing message list plus streaming.

Read first: [Doc03 — No memory](#), [Doc03 — Tokens](#), [Doc04 — Streaming](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do:

1. Add Doc04's `stream_message(client, history, user_input)` to `models/llm.py` — with `flush=True` on the print, so each piece appears the moment it arrives.
2. Write `prompts/concierge.txt` (the assistant's personality) and `utils/prompts.py` with `load_prompt(name)`.
3. Write `utils/history.py`: `estimate_tokens(history)` (characters ÷ 4, Doc03's rule) and `trim_history(history, max_tokens)` — drops the oldest turn, never the system prompt.
4. Change `main.py` into a loop: read input, `/quit` to exit, trim, stream the reply.
5. Write `tests/unit/test_history.py` and run it: `python tests/unit/test_history.py`.
6. Test memory by hand: say your name, chat about something else, then ask "what's my name?".

Your files after Step 2:

```

SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── (Setup's root files)
├── src/supportdesk/
│   ├── main.py           (changed) a chat loop with memory
│   ├── config.py
│   ├── exceptions.py
│   ├── models/llm.py     (changed) + stream_message()
│   ├── prompts/
│   │   └── concierge.txt  (new) the assistant's system prompt
│   └── utils/
│       ├── logger.py
│       ├── history.py     (new) estimate_tokens(), trim_history()
│       └── prompts.py     (new) load_prompt(name)
└── tests/unit/
    └── test_history.py    (new) runs with no API key

```

Why these files:

- `prompts/concierge.txt` — the prompt is text, not code; you can reword it without touching Python.
- `utils/prompts.py` — one function that reads a prompt file by name, so no agent opens files itself.
- `utils/history.py` — trimming is its own small job, easy to test without the API.
- `tests/unit/test_history.py` — proves trimming never drops the system prompt, in under a second.

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 3 of 4

- **Where you are now:** after Step 2 you have a working chat with memory and streaming — but it can't file a ticket, and one rate limit or too-long message crashes it.
- **What's new in this step:** `schemas.py` (`SupportTicket`), the first agent `agents/triage.py` with `prompts/triage.txt`, a temporary `/extract` command, one `except` per failure in `main.py`, sample data, a script, and tests.
- **What stays the same:** the chat loop, `models/llm.py`, and everything in `utils/`.
- **Why:** free text has to become clean, checked data that code can trust — and a real app must survive every common failure without losing the conversation.
- **How:** `/extract <text>` → `triage_agent()` → one structured-output call → a `SupportTicket` or `None`. Every OpenAI error is caught in `main.py` and gets the reaction Doc04's error table gives it.

- **Working after this step:** the chat app from Step 2, plus `/extract` to file a ticket, and it no longer crashes on API errors.

When you'll hit this for real: any feature where free text must become clean data — tickets, forms, intake.

Read first: [Doc04 — Structured output](#), [Doc04 — Error types](#), [Doc02 — JSON](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do:

1. Write `SupportTicket` in `schemas.py`, exactly as in the Real Example. It's the contract between the model's reply and your code.
2. Write `prompts/triage.txt` (strict: never guess a name) and `agents/triage.py` with `triage_agent(client, message) -> SupportTicket | None`. Use `client.beta.chat.completions.parse(...)` like Doc04, check `refusal` first, and return `None` on `ValidationError`.
3. In `main.py`, add `/extract <text>` (a temporary manual switch — Step 4 replaces it) and a `print_ticket()` helper.
4. Handle the other 3 failures in the loop: bad key (`AuthenticationError` → exit), rate limit (`RateLimitError` → ask the user to wait), too long (`BadRequestError` → drop that message). The 4th, bad structured output, is handled inside `triage_agent()`.
5. Save the 3 inputs in `data/sample/messages.json`, and write `scripts/triage_samples.py`, `tests/unit/test_schemas.py` and `tests/integration/test_live_agents.py`.
6. Run all three. Check input 2 gives `customer_name=None`.

Your files after Step 3:

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├─ (Setup's root files)
├─ src/supportdesk/
│   ├── main.py                (changed) + /extract, all error handling
│   ├── config.py
│   ├── exceptions.py
│   ├── schemas.py             (new) SupportTicket
│   ├── models/llm.py
│   ├── agents/
│   │   ├── __init__.py        (new)
│   │   └─ triage.py           (new) triage_agent()
│   ├── prompts/
│   │   ├── concierge.txt
│   │   └─ triage.txt          (new)
│   └─ utils/
│       └─ logger.py
```

```

|       | history.py
|       | prompts.py
|       |
|       | tests/
|       | | unit/
|       | | | test_history.py
|       | | | test_schemas.py (new) no API key needed
|       | | |
|       | | | integration/
|       | | | | test_live_agents.py (new) real API calls
|       | |
|       | scripts/
|       | | triage_samples.py (new) prints a ticket per sample
|       |
|       | data/sample/
|       | | messages.json (new) the 3 test complaints

```

Why these files:

- `schemas.py` — at package level, not inside `agents/`, because both agents and the tests use it.
- `agents/triage.py` — the first agent gets its own file and its own prompt from day one.
- `data/sample/messages.json` — one copy of the test inputs, read by both the script and the tests.
- `scripts/triage_samples.py` — for *you* to read the tickets and judge them; a test can't judge "is this summary good?".
- `tests/unit/test_schemas.py` vs `tests/integration/test_live_agents.py` — free and instant vs. real calls; kept apart so you never spend money by accident.

Step 4 — A Router That Sends Each Message to the Right Agent Automatically (final: 2 agents)

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 4 of 4

Read first: [Doc04 — Structured output](#) (the table row "a supervisor picks the next agent"), and, as a look ahead, [Doc11 — Patterns](#) (the Sequential and Supervisor rows).

- **Where you are now:** after Step 3 you have a chat (the future Concierge) and a Triage agent — but *you* pick between them by typing `/extract`.
- **What's new in this step:** `RouteDecision` in `schemas.py`, `agents/router.py` with `prompts/router.txt`, `agents/concierge.py` (the chat code moves out of `main.py`), and `services/desk_service.py` for the router → agent flow. `main.py` gets smaller.
- **What stays the same:** `agents/triage.py`, `models/llm.py`, `utils/`, and Step 3's error handling (it moves along unchanged).
- **Why:** the program, not the user, should decide who handles each message. That's what turns one script with two modes into a real 2-agent system.
- **How:** each message → `decide()` (a small structured call that returns an agent and a reason, and logs both) → `handle_message()` calls that agent.

- **Working after this step:** the full project — type anything; chat gets a streamed reply, a complaint becomes a ticket.

When you'll hit this for real: any system with more than one kind of request, where a person shouldn't have to say which kind it is. Project 4's Supervisor is this same idea, scaled up.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do:

1. Add `RouteDecision ( agent: Literal["concierge", "triage"], reason: str )` to `schemas.py`.
2. Write `prompts/router.txt` and `agents/router.py` with `decide(client, message) -> RouteDecision`. Log every decision with its reason. If the answer can't be read, fall back to Concierge.
3. Move the chat part into `agents/concierge.py`: `start_history()` and `concierge_agent(client, history, message)`.
4. Write `services/desk_service.py`: `handle_message()` asks the router and calls the right agent; move `print_ticket()` here too.
5. Slim `main.py` down: config, key check, then a loop that calls `handle_message()` inside the same error handling. Remove `/extract`.
6. Add routing cases to `messages.json` and the tests. Run everything, then chat with a mix of small talk and complaints — and read the log for the unclear ones.

Your files after Step 4 (final): the full tree in [Project Structure](#). What changed:

```
src/supportdesk/
├─ main.py           (changed) only input, key check, error handling
├─ schemas.py       (changed) + RouteDecision
├─ agents/
│   ├─ router.py     (new) decide() -> RouteDecision
│   ├─ concierge.py  (new) start_history(), concierge_agent()
│   └─ triage.py
├─ prompts/
│   └─ router.txt    (new)
└─ services/
    ├─ __init__.py   (new)
    └─ desk_service.py (new) handle_message(), print_ticket()
```

Why these files:

- `agents/router.py` — deciding *who* handles a message is a job of its own, with its own prompt.
- `agents/concierge.py` — the chat code moves out of `main.py`, so each agent now has its own file.

- `services/desk_service.py` — the router → agent flow, free of `input()`, so a web API could reuse it later.

Final Deliverable: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — a 2-agent terminal app in a proper package layout. A Concierge holds real multi-turn conversations. A Triage agent turns any complaint into a checked `SupportTicket`. A router decides which agent handles each message, and logs why.

Why this really is multi-agent (said plainly): a simple **routed** pattern (Doc11). No tool loop yet (Project 2), no shared graph state yet (Project 3). But it is two agents with different jobs and a router choosing between them — the real, simplest definition of multi-agent. Don't describe it as more advanced than that; Project 4 is where routing becomes a real `Command`-based supervisor.

Checklist Before You Call This Done

- [] The repo folder name matches the GitHub repo name exactly
- [] Code lives in `src/supportdesk/`, installed with `pip install -e .`; runs with `python -m supportdesk.main`
- [] Every folder has one clear job, and you can say why it exists
- [] Multi-turn conversation with real memory (Concierge)
- [] Doc01's config/logger — no secrets in code, `print()` only for what the user should see
- [] Triage tested on all 3 sample inputs, including the missing-name case
- [] The router picks the agent automatically — no `/extract` left
- [] Every routing decision is logged, with the agent and the reason
- [] Survives: bad key, rate limit, too-long message, bad structured output
- [] `tests/unit/` pass with no API key; `tests/integration/` pass with one
- [] `.env` is not in git (`git status` doesn't show it)
- [] You can explain every line of your own code, without help

Publish It to GitHub

1. Copy [README.md](#) from this folder into your repo folder, and fill in your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status          # .env and .venv/ must NOT appear
git add .
git commit -m "SupportDesk AI: 2-agent chat and triage"
```

3. On GitHub, create a new repo named exactly `SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of [README.md](#) — it's what people see in search results and on your profile. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say "**Show me the solution.**"